

Puesta en marcha

Cómo dejar corriendo en tu computador la base de datos de audios y sensores. Siete pasos, cada uno con su comprobación. Si quieres entender *por qué* está armado así, eso está en `DOCKER.md`.

145 grabaciones **36 h** de audio **15 min** por archivo · 44.1 kHz · 24 bit
2 160 lecturas de sensor

Qué es esto

Una base de datos con la **información** de los audios: cuánto dura cada uno, cuánto pesa, de qué estación y qué día es, la temperatura y la humedad que había en ese momento, y el link para escucharlo.

Los `.wav` no se guardan aquí — siguen en OneDrive. Esta base guarda los datos *sobre* ellos, que es lo que permite buscar y filtrar sin descargar 16 GB cada vez.

Antes de empezar

UNA SOLA VEZ

Docker Desktop

Corre la base de datos.

docker.com

Python 3.11+

Corre el script que carga los datos.

python.org — marca *Add Python*
to *PATH*

ffmpeg

Lee duración, formato y frecuencia de cada audio.

```
winget install Gyan.FFmpeg
```

Git

Baja el proyecto y sus actualizaciones.

git-scm.com

Cierra y vuelve a abrir la terminal después de instalar, y comprueba que Windows los encuentra:

```
docker --version
python --version
ffprobe -version
```

COPIAR

OJO Si alguno dice “no se reconoce como un comando”, no quedó en el PATH. Reinstálalo marcando la opción de agregarlo, o reinicia el computador.

1 Bajar el proyecto

HECHO

```
git clone <url-del-repositorio>
cd Proyecto-git\db
```

COPIAR

Todo lo demás se hace desde esa carpeta `db`.

2 Tener los audios en tu disco

HECHO

LÉELO Este es el paso donde todo el mundo se atasca.

El script lee archivos **locales**. Abrir la carpeta en el navegador no sirve: tiene que verse en el Explorador de Windows como una carpeta normal.

- Abre la app de **OneDrive** — el ícono de nube, abajo a la derecha.
- Engranaje → **Configuración** → pestaña **Cuenta** → **Agregar una cuenta**.
- Inicia sesión con tu correo **de la universidad**. Tu OneDrive personal y el institucional son cuentas distintas y se sincronizan por separado: si solo ves una carpeta llamada `OneDrive` a secas, esa es la personal y los audios no están ahí.
- Abre la carpeta del semillero en el navegador y dale **Sincronizar** (o *Agregar acceso directo a Mis archivos*).

Aparece una carpeta nueva, algo como `C:\Users\tú\OneDrive - Universidad\...\AudiosSemilleroCABALAS`.

COMPROBACIÓN Ábrela en el Explorador. Debes ver adentro las carpetas de estación (Estación 1 ...), y dentro de cada una los `.wav` y los `.txt`.

ESPACIO Cada grabación pesa ~113 MB. Si OneDrive los deja “solo en la nube”, la primera carga los descarga y tarda. Para evitar sorpresas: clic derecho en la carpeta → **Conservar siempre en este dispositivo**.

3 Configurar tu `.env`

HECHO

```
copy .env.example .env
notepad .env
```

COPIAR

Cambia **dos** líneas:

```
POSTGRES_PASSWORD=semillero2026
CARPETA_PROYECTO=C:\Users\TuUsuario\OneDrive - Universidad\...\AudiosSem:
```

COPIAR

Tres reglas que evitan casi todos los problemas:

- **La contraseña, solo letras y números.** Nada de `$`, `#`, espacios ni comillas: Docker y Python los interpretan distinto y después la base te rechaza la clave.
- **Elígela ahora y no la cambies más.** La contraseña queda grabada dentro de la base la primera vez que se crea. Cambiarla luego en el `.env` no cambia la base — solo hace que dejen de coincidir.
- **CARPETA_PROYECTO apunta a la carpeta que contiene las estaciones**, no a una estación. Sin comillas y sin barra al final.

RECUERDA El `.env` es tuyo y nunca se sube a git. Cada quien tiene el suyo, con su propia ruta y su propia clave.

4 Levantar la base

HECHO

Con Docker Desktop **abierto** — espera a que el ícono de la ballena deje de moverse:

```
docker compose up -d --build
```

COPIAR

La primera vez tarda varios minutos: está descargando PostgreSQL. Después:

```
docker compose ps
```

COPIAR

COMPROBACIÓN `audios-db` debe decir **healthy**. Si dice `starting`, espera 20 segundos y repite. Si dice `exited`, mira `docker compose logs db`.

5 Comprobar que quedó bien

HECHO

```
docker compose exec db psql -U audios_admin -d audios -c "\dt"
```

COPIAR

COMPROBACIÓN Deben salir **7 tablas**: `almacenamiento`, `anotacion`, `audio`, `estacion`, `etiqueta`, `fuentes`, `lectura_sensor`.

Si dice *“Did not find any relations”*, la base se creó vacía. Se arregla rehaciendo el volumen:

```
docker compose down -v  
docker compose up -d --build
```

COPIAR

CUIDADO `-v` borra los datos. Es seguro ahora que la base está vacía; **no** lo corras cuando ya tengas anotaciones hechas.

6 Cargar los datos

HECHO

```
pip install -r requirements.txt  
python ingesta.py --dry-run
```

COPIAR

`--dry-run` no escribe nada: solo muestra lo que encontró.

COMPROBACIÓN Debe listar tus audios con su duración, y tus sensores con humedad (%) y temperatura (°C) — cada `.txt` trae 15 lecturas, una por minuto.

Si se ve bien, carga primero los sensores. Son instantáneos:

```
python ingesta.py --solo sensores
```

COPIAR

Y después los audios. Este lee todos los archivos para calcular su huella, así que con 16 GB tarda varios minutos. Imprime una línea por archivo, así que vas viendo el avance:

```
python ingesta.py --solo audio
```

COPIAR

Puedes correrlo las veces que quieras: lo que ya está registrado se omite.

7 Usarlo

HECHO

Ver los links de los audios

```
python ingesta.py --links
```

COPIAR

Revisar que tu copia local y la base sigan coincidiendo

```
python ingesta.py --verificar
```

COPIAR

Consultar desde la terminal

```
docker compose exec db psql -U audios_admin -d audios
```

COPIAR

Adentro: `\x` para que la salida se lea vertical, `\q` para salir. Hay consultas listas para copiar en `ejemplos/consultas_ejemplo.sql`.

Consultar sin terminal

Abre localhost:8080 y entra con motor **PostgreSQL**, servidor **db** — el nombre del servicio, no `localhost` — y el usuario, la clave y la base de tu `.env`.

LO QUE VES	QUÉ PASA
no se reconoce como un comando	Falta el PATH. Reinstala marcando “Add to PATH” y reabre la terminal.
cannot connect to the Docker daemon	Docker Desktop está cerrado. Ábrelo y espera a que arranque.
port is already allocated	Algo más usa el 5433. Cámbialo a 5434:5432 en <code>docker-compose.yml</code> y pon <code>PGPORT=5434</code> en el <code>.env</code> .
password authentication failed	El <code>.env</code> y la base no coinciden. Se arregla abajo, sin borrar nada.
esta carpeta no existe en este equipo	<code>CARPETA_PROYECTO</code> está mal puesta. El propio error te lista las carpetas de OneDrive que sí encuentra.
la estación 'X' no está en la tabla estacion	El nombre de la carpeta no coincide con la tabla. Se arregla abajo.
Did not find any relations	La base se creó vacía. <code>docker compose down -v</code> y vuelve al paso 4.
ingesta.py parece colgado	No lo está: está leyendo gigabytes de audio. Déjalo trabajar.

Arreglar la contraseña sin borrar nada

```
docker compose exec db psql -U audios_admin -d audios -c "ALTER USER audios_admin PASSWORD 'audios_admin';"
```

Y pon exactamente esa misma en el `.env`. Funciona sin pedirte clave porque `exec` entra desde dentro del contenedor.

Agregar una estación nueva

Si aparece una carpeta `Estación 2`, la base tiene que saber que existe:

```
docker compose exec db psql -U audios_admin -d audios -c "INSERT INTO estacion (nombre, descripcion) VALUES ('Estación 2', 'Estación 2');"
```

EXACTO El nombre debe ser idéntico al de la carpeta: tildes, espacios y mayúsculas. Agrega también esa línea a `sql/02_semillas.sql` y súbela a git, para que a los demás les quede creada de una.

Reglas del equipo

- ✗ **Nunca subas tu `.env` a git.** Tiene tu contraseña. Ya está en el `.gitignore` ; no lo saques de ahí.
- ✗ **No renombres las carpetas de estación** en OneDrive sin avisar. La base las identifica por nombre exacto y la ingesta deja de reconocerlas.
- ✗ **No corras `docker compose down -v` con anotaciones cargadas.** Borra los datos. Para apagar sin perder nada: `docker compose stop`.
- ✓ **Antes de tocar algo pesado, respalda:**
`docker compose exec db pg_dump -U audios_admin audios > respaldo.sql`

Comandos de uso diario

```
docker compose up -d      # encender
docker compose stop       # apagar (los datos quedan)
docker compose ps         # ver estado
docker compose logs -f db # ver qué está pasando (Ctrl+C para salir)

python ingesta.py         # cargar lo nuevo
python ingesta.py --verificar # revisar que todo cuadre
python ingesta.py --links  # ver los links
```

COPIAR

Semillero CABALAS El porqué de cada decisión: DOCKER.md

Consultas de ejemplo: ejemplos/consultas_ejemplo.sql

